Data Challenge, Prerequisites, Environment Setup

DAIR^3

Jackson State U · U Michigan · UT San Antonio

DAIR^3 Team

Faculty:
Clifton Addison, Associate Professor of Biostatistics, Jackson State University
Yalanda Barner, Assistant Professor of Health Policy and Management, Jackson State University
Johann Gagnon-Bartsch, Associate Professor of Statistics, University of Michigan
Juan B. Gutiérrez, Principal Investigator; Professor of Mathematics, University of Texas at San Antonio
Gregory Hunt, Assistant Professor of Mathematics, College of William and Mary
Brenda Jenkins, Director of Training and Education, Jackson State University
Erin Kaleba, Director, Data Office for Clinical and Translational Research, University of Michigan
Jing Liu, Principal Investigator; MIDAS Executive Director, University of Michigan
Jodyn Platt, Associate Professor of Learning Health Sciences; Associate Professor of Health Management and Policy, University of Michigan Medical School
Suraj Rampure, Lecturer III in Electrical Engineering and Computer Science, University of Michigan
Kerby Shedden, Professor of Statistics; Professor of Biostatistics, University of Michigan
Teaching Assistants:
Taofeq... complete
Sim... complete
Admisnitrative Support:
Kelly Psilidis, Faculty Training Program Manager, University of Michigan
Michele Randolph, Evaluation Specialist, Marsal School of Education, University of Michigan

Document prepared by: Juan B. Gutiérrez, Ph.D. Professor of Mathematics, juan.gutierrez3@utsa.edu

The University of Texas at San Antonio

March 2, 2026

Contents

1 2026 Data Challenge - Vital Statistics **4**
- 1.1 Problem Description . . . 5
- 1.2 Data Description . . . 5
- 1.3 Identifying Underweight Cutoff Points . . . 6
- 1.4 Data Files . . . 8
- 1.5 Criteria . . . 8

2 Foundational Analysis Activities **10**
- 2.1 A single year of vital statistics . . . 11

3 Environment Setup **13**
- 3.1 Required: Setting up environment variables for API keys . . . 13
- 3.2 Required: Install Git . . . 14
- 3.3 Required: Open Git Bash (or any terminal) . . . 15
- 3.4 Required: Install Python . . . 15
- 3.5 Optional: C/C++ compilers . . . 16
- 3.6 Optional: LaTeX . . . 17
- 3.7 Strongly recommended: Install Visual Studio Code (VSC) . . . 17
- 3.8 If you installed VSC: Required extensions . . . 17
- 3.9 Optional: Windows Terminal . . . 18
- 3.10 If you installed VSC: Required . . . 19
- 3.11 Partially Required (this or 3.12): Virtual environments inside VS Code . . . 19
- 3.12 Partially Required (this or 3.11): Virtual environments outside VS Code . . . 23
- 3.13 Troubleshoot . . . 25
- 3.14 Required (after 3.11 or 3.12): Install Python libraries . . . 25
- 3.15 Required: Jupyter Notebook . . . 26
- 3.16 Python commands in Terminal . . . 27
- APPENDIX: ActivateEnv.bat . . . 28

4 Introduction to Programming Environments **30**
- 4.1 Introduction . . . 30
  - 4.1.1 Compiled Languages . . . 30
  - 4.1.2 Scripting Languages . . . 31
  - 4.1.3 Comparative Examples . . . 32

4.1.4 What is Python? 33
4.1.5 Python Installation 33
4.1.6 Executing Python Commands 34
4.1.7 The VSC Python Editor: Executing A Simple Program . . . 35
4.1.8 Executing Functions 38
4.2 Introduction to NumPy 39
4.2.1 Matrix Operations 40

Chapter 1

2026 Data Challenge - Vital Statistics

1.1 Problem Description

Your team is part of a consultancy supporting a planning commission for the State of Texas. The commission is planning budgetary requirements for various State services in 2030. The commission requests the following:

- Projections of underweight newborns by county in Texas.
- Projections of newborn mortality by county in Texas. According to the CDC, a stillbirth is classified as either early, late, or term. An early stillbirth is a fetal death occurring between 20 and 27 completed weeks of pregnancy. A late stillbirth occurs between 28 and 36 completed pregnancy weeks. A term stillbirth occurs between 37 or more completed pregnancy weeks.
- Identification of the socioeconomic factors associated with these two outcomes.
- Comparison to other states.

Your team will produce an executive report accompanied by an appendix of technical material.

1.2 Data Description

The commission has approved specific data sources for this analysis. You can only use approved data available at this hyperlink.

National Center for Health Statistics (NCHS): Since 1969, all births recorded in the US are available as digital records from the NCHS, from the Centers for Disease Control and Prevention (CDC). The date and time of birth is publicly available only between 1969 and 1988; starting in 1989, only the week of birth is recorded. The NCHS keeps records of place of birth, assistance during delivery (at home, with doctors, with midwives), level of education of the parents, place of residence, weight at birth, number of weeks of gestation, number of siblings, birth order, etc. In total, over 100 variables are recorded. You have access to records form 1969 to 1988.

Surveillance, Epidemiology, and End Results Program (SEER): The U.S. Census Bureau annually releases unabridged population estimates for five-year age groups and race at the county level. The Census Bureau does not release bridged race estimates by single year of age at the county level due to concerns about the reliability of these estimates. However, these estimates are provided to the National Cancer Institute through SEER to meet programmatic needs such as the creation of age groupings that differ from the standard groupings used by the Census Bureau. Users of the single-year-of-age county-level interpolated race population estimates should carefully consider the limited reliability of these estimates. County-level population files with 19 age groups (<1, 1-4, ..., 80-84, 85+) and with 86 single-year age groups (<1, 1, 2, ..., 84, 85+) are provided.

Socioeconomic Data and Applications Center (SEDAC): SEDAC, the Socioeconomic Data and Applications Center, is one of the Distributed Active Archive Centers (DAACs) in the Earth Observing System Data and Information System (EOSDIS) of the U.S. National Aeronautics and Space Administration (NASA). SEDAC contains georeferenced U.S. county-level population projections, total and by sex, race and age, based on shared socioeconomic pathways (SSPs). This data set, produced by Mathew E. Hauer, consists of county-level population projection scenarios of population in five-year intervals for all U.S. counties for the period 2020 - 2100. Obtain the data description from https://doi.org/10.1038/sdata.2019.5.

The NCHS data set covers from 1969 through 1986; it provides individual birth data. The SEER data set provides age-bracketed population estimates from 1969 to 2020. Since for this exercise we do not have birth data beyond 1986, you will have to use the SEER data to infer births in the period 1987-2020; alternatively, you could go to the NCHS source and obtain more recent data, however the NCHS data is complex and downloading additional years is not advised in the short time available to complete the Rowdy Datathon (but we will not stop you). The SEDAC data has total population estimates per county from 2020 through 2100 categorized in four ethnicities: Hispanic, white, black, and other. A viable sequence of analysis is NCHS → SEER → SEDAC.

1.3 Identifying Underweight Cutoff Points

The CDC offers a data table of infant weight for age, available at:
https://www.cdc.gov/growthcharts/data/zscore/wtageinf.xls
You can extract from this table the information related to weight at birth.

The Excel file `wtageinf.xls` contains the LMS parameters and selected percentile values needed to compute exact percentiles and z-scores for anthropometric measurements. The parameters are provided by sex (1 = male, 2 = female) and by single month of age. Age is listed at the half-month point representing the entire month (e.g., 1.5 months corresponds to 1.0–1.99 months). The only exception is birth, which represents the point at birth.

The LMS method summarizes the distribution of a measurement at a given age using three parameters:

- L: the Box–Cox power transformation,
- M: the median,
- S: the generalized coefficient of variation.

These parameters allow computation of both percentiles and z-scores.

To obtain the measurement value X corresponding to a given z-score Z (or percentile), use:

$$X = M(1 + LSZ)^{1/L}, \quad L \neq 0$$

$$X = M\exp(SZ), \quad L = 0$$

where L, M, and S are taken from the Excel row corresponding to the child's age (in months) and sex. The z-score corresponding to common percentiles is:

$$\begin{aligned} -1.881 &\leftrightarrow 3^{\text{rd}} \\ -1.645 &\leftrightarrow 5^{\text{th}} \\ -1.282 &\leftrightarrow 10^{\text{th}} \\ -0.674 &\leftrightarrow 25^{\text{th}} \\ 0 &\leftrightarrow 50^{\text{th}} \end{aligned}$$

Example: For a 9-month-old male, the `WTAGEINF` table gives:

$$L = -0.1600954, \quad M = 9.476500305, \quad S = 0.11218624.$$

Using $Z = -1.645$ (5th percentile), the cutoff is:

$$X = 7.90 \text{ kg}.$$

This value represents the 5th percentile weight-for-age and can be used as an underweight threshold if the 5th percentile definition is adopted.

Computing a Z-Score from a Measurement: To obtain the z-score corresponding to a given measurement X:

$$Z = \frac{(X/M)^L - 1}{LS}, \quad L \neq 0$$

$$Z = \frac{\ln(X/M)}{S}, \quad L = 0.$$

The corresponding percentile is then obtained from the standard normal distribution.

Example: For a 9-month-old male weighing 9.7 kg:

$$Z = 0.207,$$

which corresponds approximately to the 58th percentile.

Identifying Underweight Cutoff Points in Excel: To identify an underweight threshold in the Excel file:

1. Filter the data by sex (1 = male, 2 = female).
2. Locate the row corresponding to the child's age in months.
3. Identify the desired cutoff percentile (e.g., 5th or 3rd percentile column).

4. Alternatively, compute the exact cutoff using the LMS formula above.

A child is classified as underweight if their measured value falls below the selected percentile cutoff (e.g., below the 5th percentile or below $Z = -2$, depending on the chosen clinical definition).

If finer age resolution is required, linear interpolation between adjacent months may be applied to the L, M, and S parameters prior to computation.

1.4 Data Files

Table 1.1: File sizes. If you have limited storage, plan your analysis in stages.

DATA SET	FILE	ZIPPED	UNZIPPED
NCHS	US1969-1986.zip	2.7 GB	22.8 GB
	natalityConfBackup_PostgreSQL.sql	2.42 GB	25 GB
	US1969.zip	32.6 MB	381 MB
SEDAC	hauer_county_NH_pop_SSPs.xlsx	N/A	15.1 MB
SEER	SEER Data Dictionary.pdf		73 KB
	tx.1969_2020.19ages.adjusted.txt.gz	5.3 MB	35 MB
	tx.1969_2020.singleages.adjusted.txt.gz	18.8 BM	19 MB
	tx.1990_2020.19ages.adjusted.txt.gz	6.5 MB	6 MB
	tx.1990_2020.singleages.adjusted.txt.gz	20.9 MB	21 MB
	us.1969_2020.19ages.adjusted.txt.gz	66.7 MB	430 MB
	us.1969_2020.singleages.adjusted.txt.gz	238.8 MB	1.6 GB
	us.1990_2020.19ages.adjusted.txt.gz	76.7 MB	520 MB
	us.1990_2020.singleages.adjusted.txt.gz	246.3 MB	1.8 GB
TOTAL		5.7 GB	52 GB

1.5 Criteria

Results will be evaluated according to requirements set by the commission:

- **Compelling presentation**: You must enable the commission to share your numerical and graphical results directly with legislators and citizens through executive summaries. This lay audience should find your summaries and implications to be understandable and convincing. Express your results in ways that can be acted on to plan e.g. funding of schools, care for the elderly, etc. The supporting documentation can be technical.

- **Analysis comprehension**: Before a single line of code is written, before a single byte of raw data is processed, you must be able to tell the story of what is the progression of steps that will be undertaken in analysis.
- **Sound technical methods**: You may cite the analyses of others, but the commission wants to see the methods that you have invented or adopted to calculate these projections (which should be accompanied by error bars, if possible). The commision must have confidence in your results in order to present those results to others.
- **Awareness of the data context**: All data have bias. Before, during and after analysis, it is essential to identify biases in the data and articulate clearly how these biases influence all steps of analysis and interpretation.
- **Reproducible results**: You must enable the commission to have your results confirmed by an independent team. That is, enable the independent team to replicate your results by describing your data and methods in detail.

Table 1.2: Evaluation Criteria

CRITERION	**% WEIGHT**
1. Compelling presentation - Informative	10%
2. Compelling presentation - Understandable	10%
3. Analysis comprehension	20%
4. Sound technical methods	20%
5. Awareness of the data context	20%
6. Reproducible results	20%

Chapter 2

Foundational Analysis Activities

2.1 A single year of vital statistics

A problem most people can relate to is demography. Millions of people are born in the US every year. Recording birth events is necessary for legal matters such as obtaining a driver's license or other forms of government-issued identification. However, recording, keeping, and using this information has challenges that exemplify many aspects of data analysis, as this exercise will demonstrate.

Since 1969, all births recorded in the US are available as digital records from the National Center for Health Statistics (NCHS) from the Centers for Disease Control and Prevention (CDC). Only between 1969 and 1988 the date and time of birth is publicly available; starting on 1989, only the week of birth is recorded. Why is the date and time of birth no longer recorded? Record your explanation as answer #1.

The NCHS keeps records of place of birth, assistance during delivery (at home, with doctors, with midwives), level of education of the parents, place of residence, weight at birth, number of weeks of gestation, number of siblings, birth order, etc. In total, over 100 variables are recorded.

I have made available two files for you: a compressed file with birth records from 1969, and a data dictionary. The uncompressed data file is about 380 MB in size. The data is contained in a "flat file". This means that every line of text in this file is a continuous chain of characters. We must extract information from this type of files with a "dictionary" that tells us the beginning and ending columns of a given variable. Why was this format used? Record your explanation as answer #2.

Please answer the following questions:

1. How many live births occurred in Texas in 1969 from mothers residing in Texas? Record your explanation as answer #3.

    - Bonus question: How would you visualize births from each state with respect to every other state?

2. Show graphically how the level of education of the mother is related to the birth order (1st born, second child, third, etc.) Record your explanation as answer #4.

    - Bonus question: How would you visualize each variable with respect to every other variable?

It is possible that you might not know how to answer some of these questions on first contact with this problem.

A simple program that can help you explore this file is

```
f = open("US1969.dat", "r", encoding="cp1252")
counter = 0;
for x in f:
    if x[0] == "9" and [... other conditions?]
        counter = counter + 1
print(counter)
```

Note that in Python indexing is subtle. For example:

```
x = [10, 20, 30, 40, 50]
x[1:3]  # → [20, 30]  (indices 1 and 2, not 3)
```

The instruction *encoding="cp1252"* is necessary in non-Windows systems due the presence of single-byte character encoding of the Latin alphabet, used by default in the legacy components of Microsoft Windows for English and many European languages including Spanish, French, and German. If you remove this instruction, the following error might show up in non-Windows operating systems: "'utf-8' codec can't decode byte..."

A common approach to extract information from flat files is by importing it into Excel. The "*Text Import Wizard*" (typically shown as in Figure 2.1-1) would guide you through the process of identifying variables by column. However, this results in a problem. Describe it. Record your explanation as answer #5.

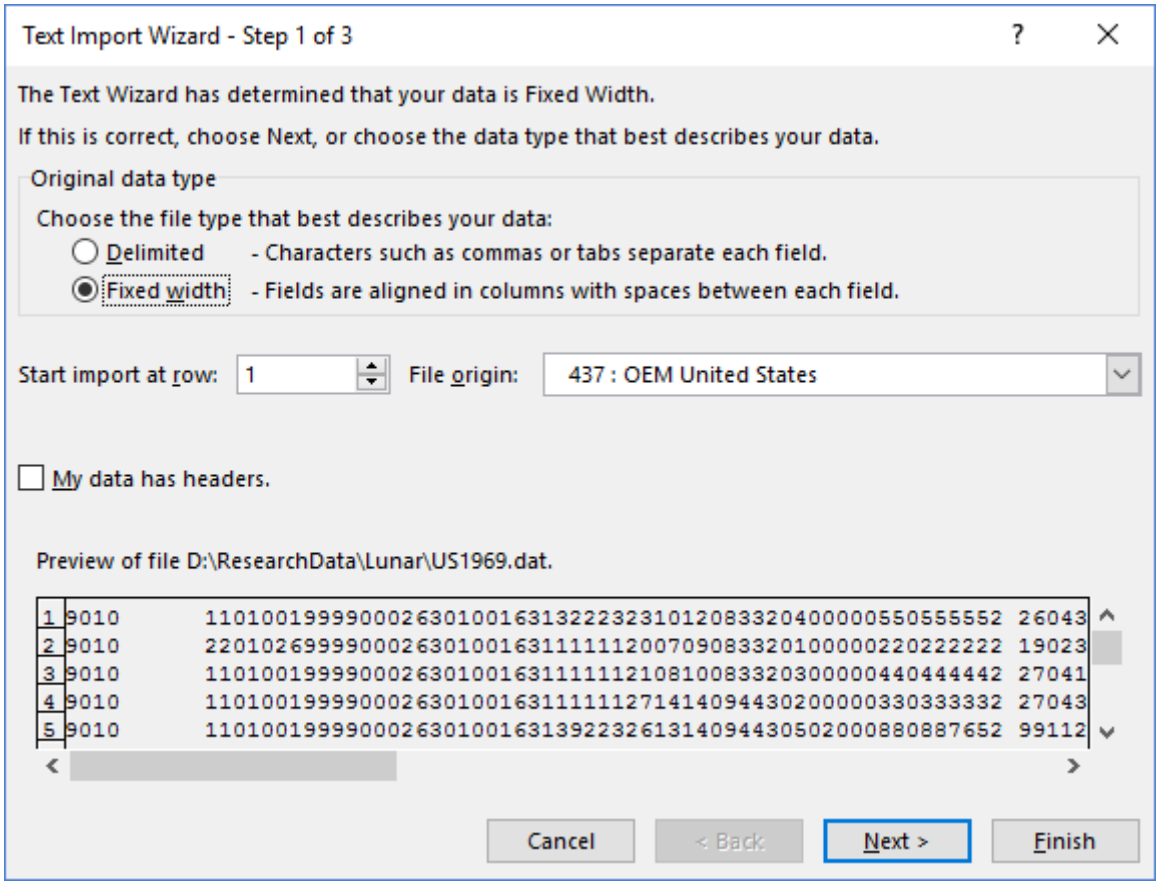

Figure 2.1-1: Excel's Import Wizard

Chapter 3

Environment Setup

These instructions centered on Windows but adaptable for Mac and Linux. We begin with configuring environment variables to securely store API keys for OpenAI, Anthropic, and Groq. The reader is then guided through the installation of Python, emphasizing proper PATH configuration and optional tools like Visual Studio Code (VSC), Git, and Windows Terminal. Additional sections cover the setup of virtual environments both within and outside VSC, the installation of essential Python libraries, and optional tools such as LaTeX and C/C++ compilers. The setup culminates in verifying functionality by cloning a GitHub repository and running a helper script, ensuring the environment supports modular, reproducible project development with minimal dependency conflicts.

3.1 Required: Setting up environment variables for API keys

You will receive an API key for OpenAI and Anthropic during our first session. You can get your own API for free at Groq. You need to create three API key variables following the procedure described below: `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GROQ_API_KEY`. Test the program agentGroq.py, agentGPT.py and/or agentClaude.py from the Github repository described below in this document.

On Windows, execute the following three commands one at a time for each API key. The example for `OPENAI_API_KEY` is as follows:

```
setx OPENAI_API_KEY "[sk-... API key]"
Exit
echo %OPENAI_API_KEY%
```

In the code above, replace `[sk-... API key]` with your actual API key. The first command sets the environment variable permanently for your user account. The second command closes the Command Prompt to ensure the new environment variable is registered. The third command, run in a new Command Prompt window, prints the stored value to verify it was set correctly.

On Mac/Linux execute the following three commands one at a time for each API key. The example for `OPENAI_API_KEY` is as follows:

```
echo "export OPENAI_API_KEY='[sk-... API key]'" >> ~/.zshrc
source ~/.zshrc
echo $OPENAI_API_KEY
```

In the code above, replace `'[sk-... API key]'` with your actual API key.

As reported by Bryan Fowler, if you experience errors trying to add your environmental variables in a Mac/Linux, the most likely problem is ownership of the `~/.zshrc` file. In this case, execute

```
sudo chown $(whoami) ~/.zshrc
```

And try to create the environmental variables again.

As reported by Drew Stephen regarding Mac, "*apparently it doesn't work on the default c shell but switching to the zsh lets it run.*" You might need to close and reopen the terminal window from where you started for changes to become effective.

3.2 Required: Install Git

Install Git for Windows. For Mac, install Git for Mac. Linux also has a version available. In the second screen, select Visual Studio Code as your Git editor. Use all other default options.

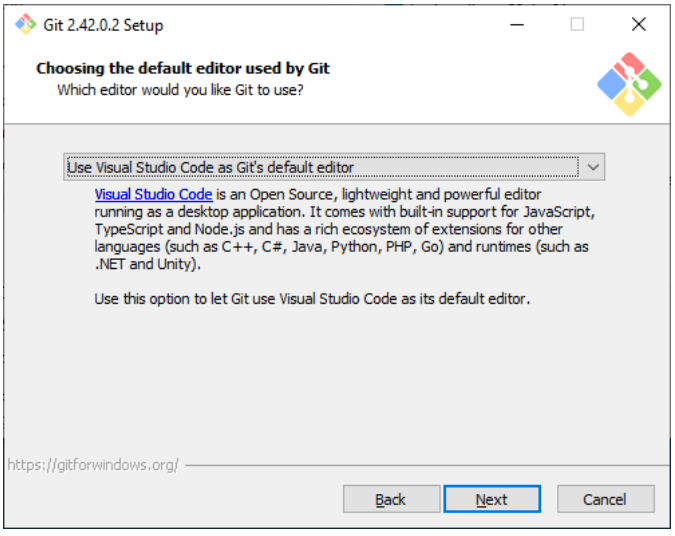

Figure 3.2-1: Git setup screen showing Visual Studio Code as the default editor.

3.3 Required: Open Git Bash (or any terminal)

The command line allows you to control your computer by typing instructions, providing precise access to files and system features. For example, typing `ls` on a Unix-like system such as Linux or macOS lists the files and directories in the current location, such as `ls /home/user/Documents` to display the contents of the Documents folder. On Windows, the `dir` command serves the same function, so `dir C:\Users\Alice\Desktop` shows what is on the Desktop. To move between directories, the `cd` command is used; `cd Downloads` changes the working directory to Downloads, while `cd ..` moves one level up. These commands allow efficient navigation and inspection of the file system through the terminal.

Now that you installed git, you should **go to the folder you want to work in** using command line, and run the following command:

```
git clone https://github.com/DiscursiveNetworks/FOO_QtPy.git
```

3.4 Required: Install Python

https://www.python.org/downloads/. Install any recent Python version. Go to your downloads and double click on the install file to start installation. By default the Python installer for Windows places its executables in the user's AppData directory, so that it doesn't require administrative permissions. This works for most scenarios. If you're the only user on the system, you might want to place Python in a higher-level directory (e.g. `C:\Python` or `/usr/local/bin`) to have a shorter path to the binaries (sometimes you will need that). Depending on your preferences, either select "*Install Now*" or "*Customized installation*" (my preference). Please make sure you select "*Add python.exe to path*"; this will save you a few headaches later.

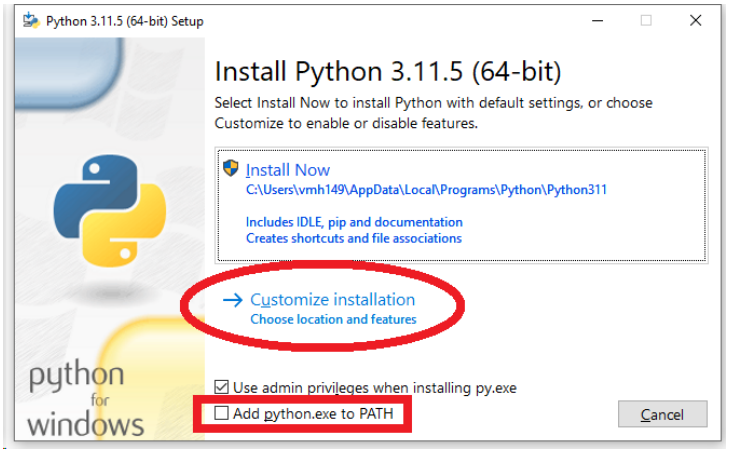

Figure 3.4-2: Python installer showing the "Add python.exe to PATH" checkbox at the bottom.

For PC: If you did not add `python.exe` to the PATH during installation, wait until the installation is complete. Then open File Explorer and right-click on "This PC." Select "Properties" from the context menu. In the System window

that opens, click on "Advanced system settings" in the left-hand sidebar. In the new window, ensure the "Advanced" tab is selected and click the "Environment Variables" button near the bottom. Under the "System variables" section, find and double-click on the variable named "Path." This will open a window where you can add the path to the Python executable manually.

For Mac: If you did not ensure Python was added to your system's PATH during installation, you can manually do so using the following steps on a Mac. After installation is complete, open the Terminal application. Enter the command `which python3` or `which python` to find the path to the installed Python binary. Copy that path. Then open your shell configuration file in a text editor—this will typically be `~/.zshrc` if you are using the Zsh shell (default on newer versions of macOS) or `~/.bash_profile` if you are using Bash. Add the line `export PATH="/path/to/python:$PATH"`, replacing `/path/to/python` with the path you copied. Save the file and close the editor. In the Terminal, run `source ~/.zshrc` or `source ~/.bash_profile` depending on the file you edited, so the new PATH is loaded into your environment.

In the window "Optional Features" select all features

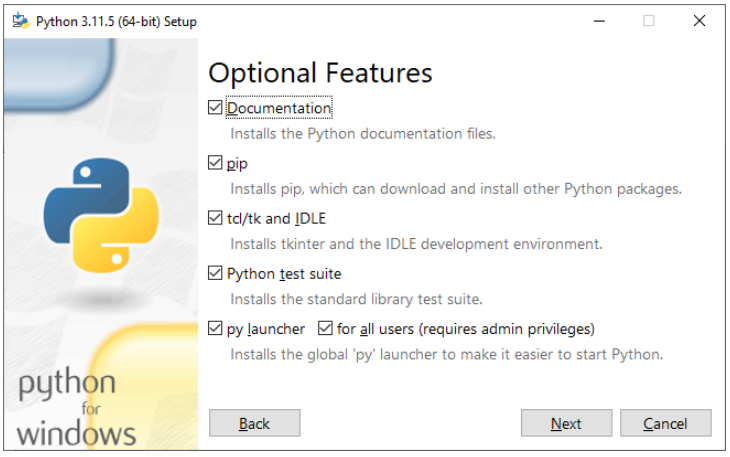

Figure 3.4-3: Python Optional Features screen—select all features.

Select "*Install Python X.XX for all users*" if you can and want. This requires administrative privileges. Select the folder of your choice for the install location.

3.5 Optional: C/C++ compilers

If you'd like to code in C or C++, you have several options, including GCC, the GNU Compiler Collection, but it requires some effort to install the prerequisites. From Microsoft there is Visual Studio Community Edition (VSCE), which encapsulates the complexity of installing multiple compilers (including the .NET framework SDK). I recommend you install VSCE with all "Workloads" in "Web & Cloud" and "Desktop & Mobile" (except Python).

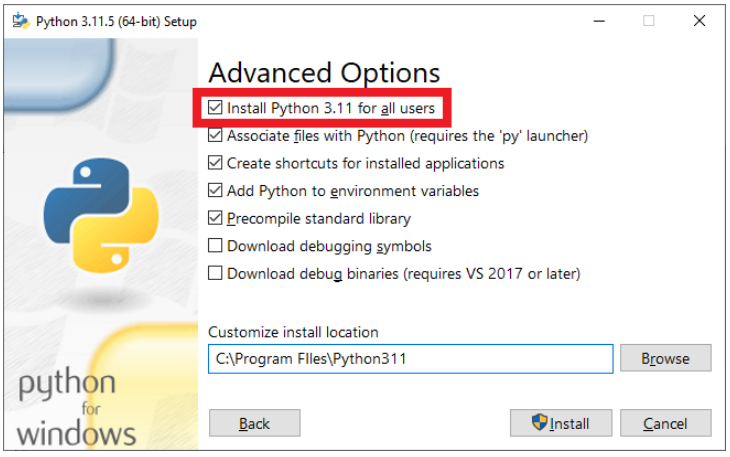

Figure 3.4-4: Python Advanced Options screen showing "Install for all users" selected.

3.6 Optional: LaTeX

If you intend to produce PDF documents using LaTeX, install MiKTeX.

3.7 Strongly recommended: Install Visual Studio Code (VSC)

If you are new to Python, install Visual Studio Code (VSC): There are many options to write Python code, and you could have a favorite one. I must pick one to deliver instruction, and experience has shown me that VSC offers the least amount of friction.

In VSC, there are two main options: "*User Installer*" and "*System Installer*". Use the User Installer if you want to install only for the current user. For all users, use System installer; I prefer this choice because it installs VSC in `C:\Program Files\Microsoft VS Code`.

This little giant has become a darling of coders for many good reasons. It is available for Mac, Linux and Windows. There is an open source identical alternative called VSCodium, available at https://vscodium.com/ I personally use VSCode.

3.8 If you installed VSC: Required extensions

Select the Extensions icon on the left and install the following Visual Studio Code libraries

1. **Required**: Python extension for Visual Studio Code.
2. **Optional**: Also install Jupyter (this also installs Pylance, Jupyter Keymap, Jupyter Notebook Renderers, Jupyter SlideShow). Pay attention to the publisher of the extension; use the ones by Microsoft.

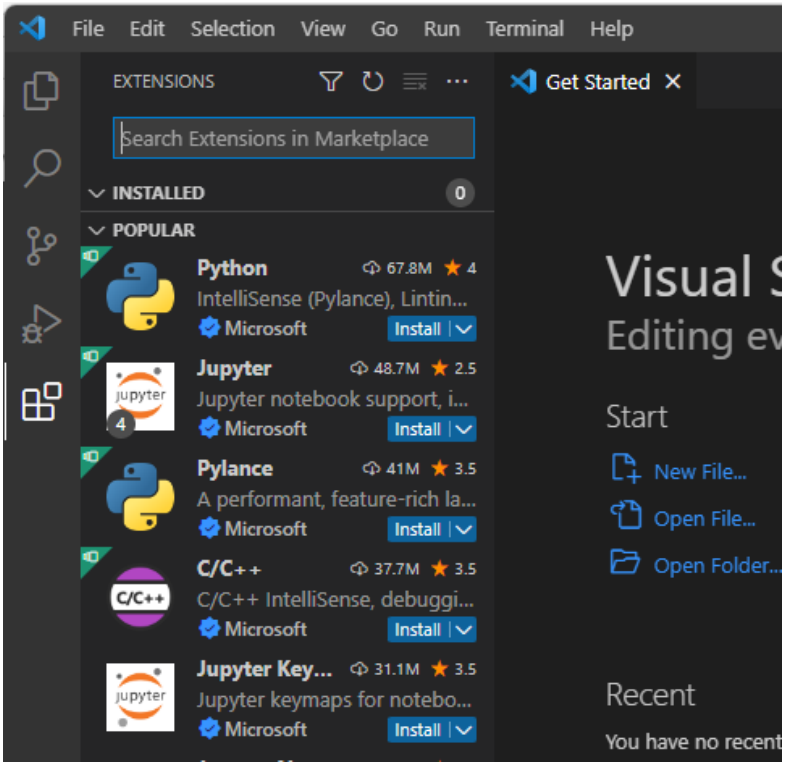

Figure 3.8-5: VS Code Extensions marketplace showing the Python and Jupyter extensions by Microsoft.

3. **Optional**: Install C/C++ for Visual Studio Code, C/C++ Themes & C/C++ Extension Pack.

4. **Optional**: Install the extension "**LaTeX Workshop**" by James Yu. You will never use another LaTeX editor :)

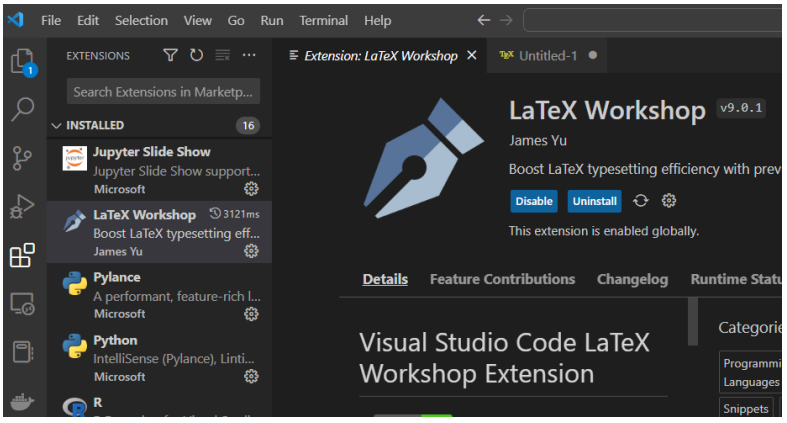

Figure 3.8-6: LaTeX Workshop extension page in VS Code.

3.9 Optional: Windows Terminal

For Windows users, install Windows Terminal. This Microsoft app allows you to use a multi document window with tabs for different command consoles like

PowerShell, Ubuntu, DOS, Git, etc. You will need this if you want to complete the steps related to Linux in the next section of this document.

3.10 If you installed VSC: Required

To start VSC, use the command line tool for your operating system and go to the folder CommandLineLLM. After Step 9, the command will be

```
cd CommandLineLLM
```

Inside this folder, execute the command

```
code .
```

Note the period after the command `code`; it instructs VSC to use the current folder as the working folder.

3.11 Partially Required (this or 3.12): Virtual environments inside VS Code

Create virtual environments *inside* Visual Studio. We will start by creating a virtual environment; the reason for this is that sometimes specific versions of different libraries might be incompatible. You should create an environment for each project you work on.

1. Open Visual Studio. Select the option *File > Open Folder...* Choose the folder you want to use for your project.
2. Once you select the folder, you will see the files related to the project you are working on in the Visual Studio File Explorer. Notice that the status bar changes color from purple to blue.
3. Now, hit the keys *SHIFT+CTRL+P*. This will allow you to type in the command palette the following instruction:

   ```
   Python: Create Environment
   ```

4. You will likely see two options: `venv` and `conda`. Choose `venv`.
5. Once you choose `venv`, you will have to pick a Python installation. Pick the one we just installed (this could be the only one you see).
6. Now, go to the menu *View > Terminal*.
7. You will see that the command prompt now has the word `(.venv)`. This means that the virtual environment has been installed and is now active. If it has not been activated, you can simply type `activate`. To inactivate the virtual environment, you simply need to type `deactivate`.

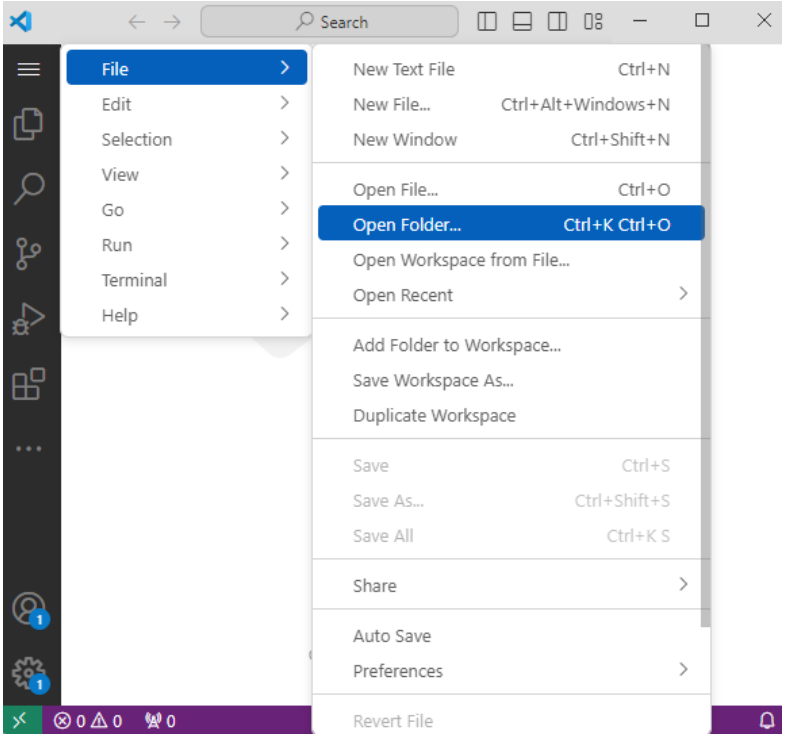

Figure 3.11-7: VS Code File menu showing Open Folder.

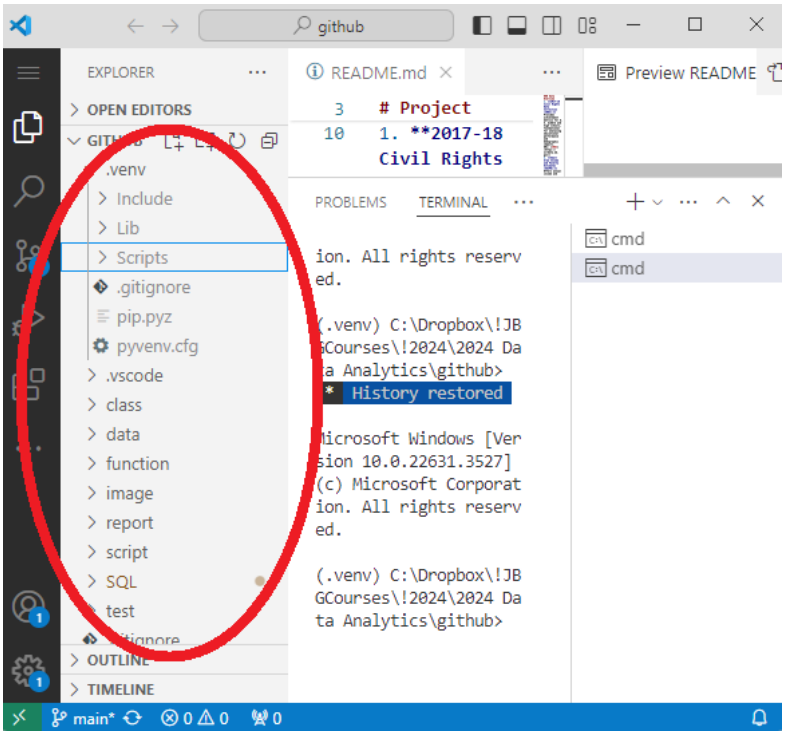

Figure 3.11-8: VS Code with a project open—notice the blue status bar.

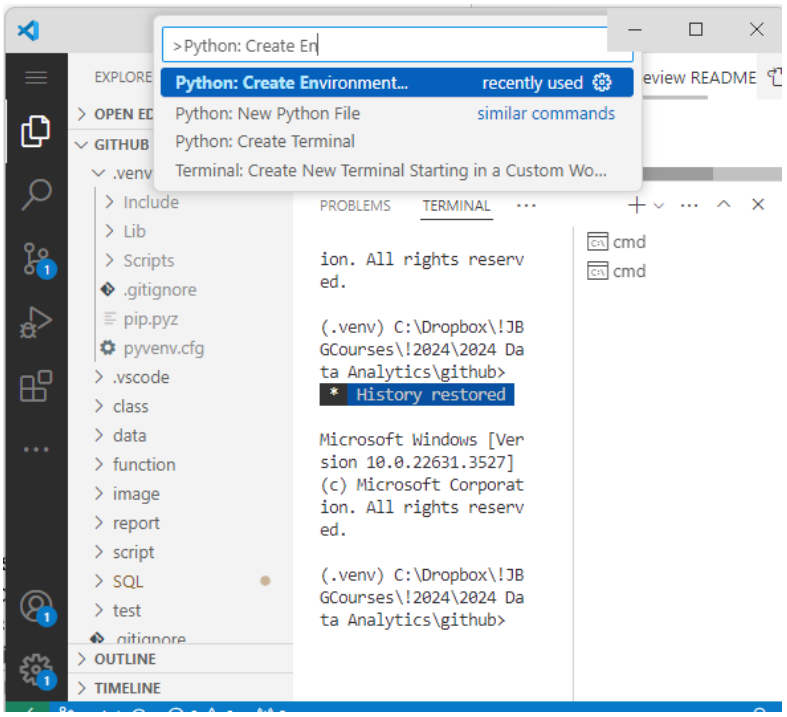

Figure 3.11-9: VS Code Command Palette with "Python: Create Environment" selected.

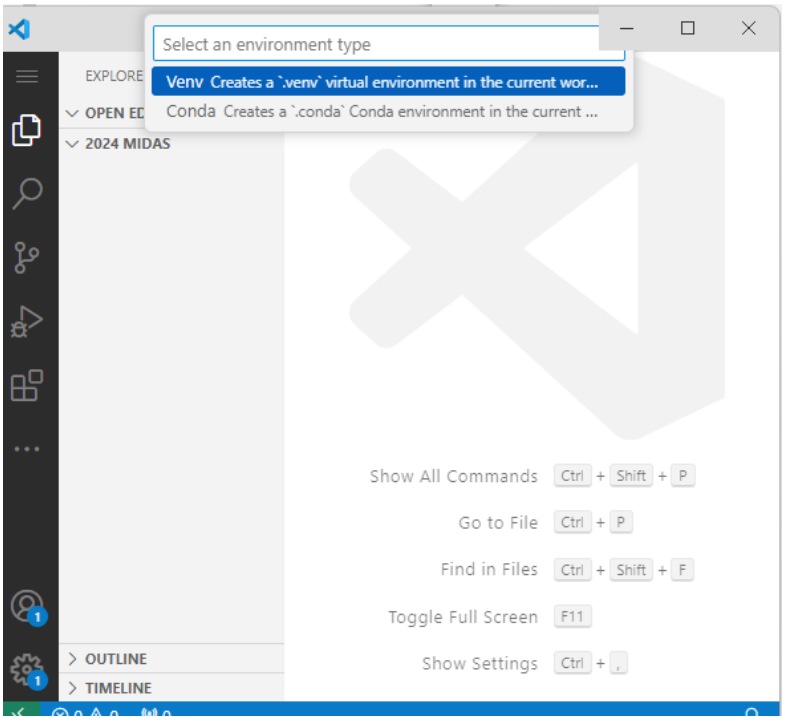

Figure 3.11-10: Selecting venv over conda in VS Code.

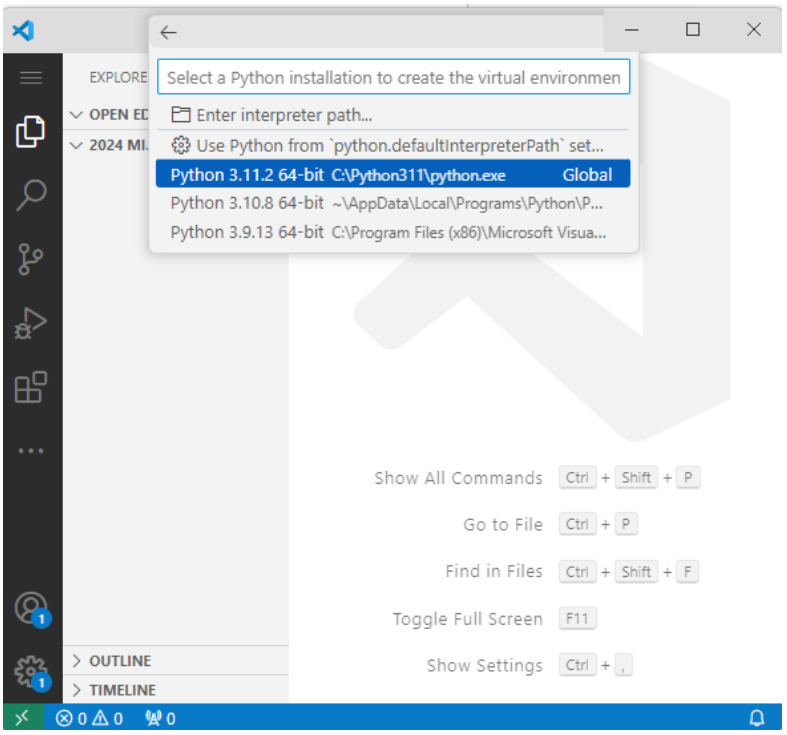

Figure 3.11-11: Selecting the Python installation for the virtual environment.

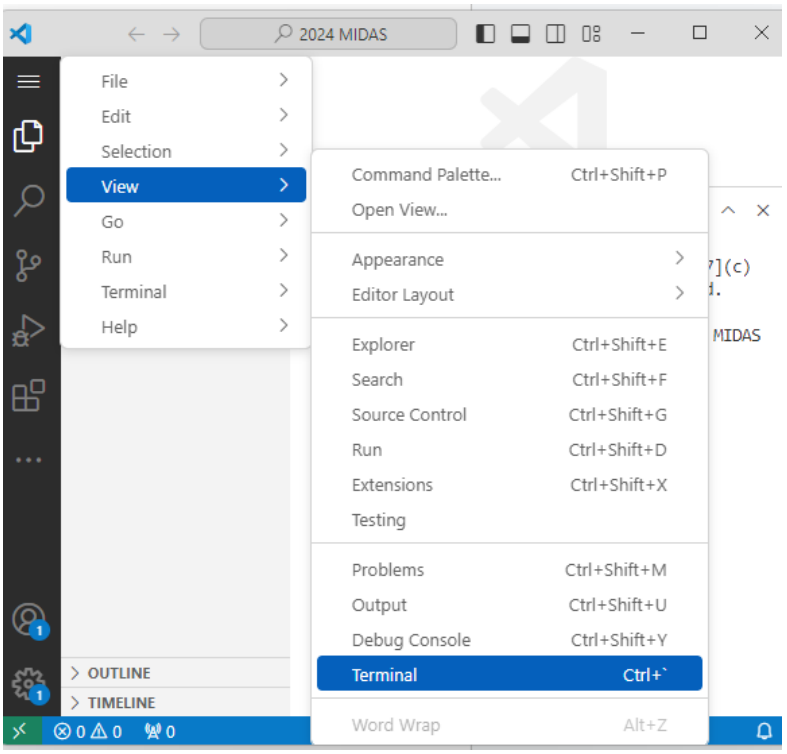

Figure 3.11-12: VS Code View menu showing Terminal option.

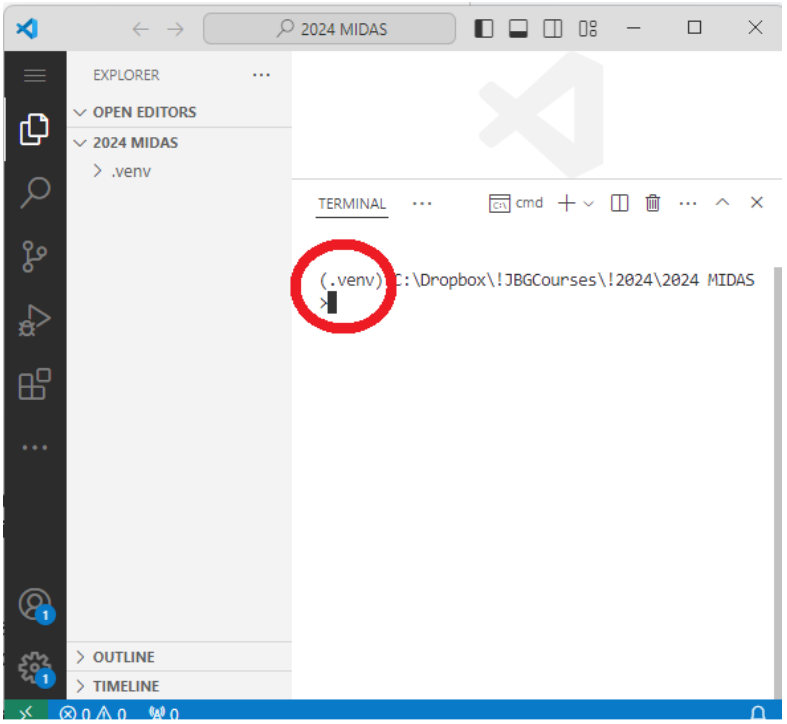

Figure 3.11-13: Terminal showing the `(.venv)` prompt indicating an active virtual environment.

3.12 Partially Required (this or 3.11): Virtual environments outside VS Code

Create virtual environments *outside* Visual Studio. A comprehensive guide is here:
https://biomathematicus.me/working-with-python-virtual-environments-in-visual-studio-code/

Open a terminal window. Now you are ready to install Python packages. We will start by creating a virtual environment; the reason for this is that sometimes specific versions of different libraries might be incompatible. You should create an environment for each project you work on.

1. All related instructions about packages and virtual environments are at:

   https://packaging.python.org/en/latest/tutorials/installing-packages/

2. Check your version of pip. It is the most popular tool for installing Python packages, and the one included with modern versions of Python. If the following command returns a version number, you are good to go. Otherwise, reinstall python (or bootstrap it).

   ```
   python -m pip --version
   ```

3. Update the pip and setuptools

```
python -m pip install --upgrade pip setuptools wheel
```

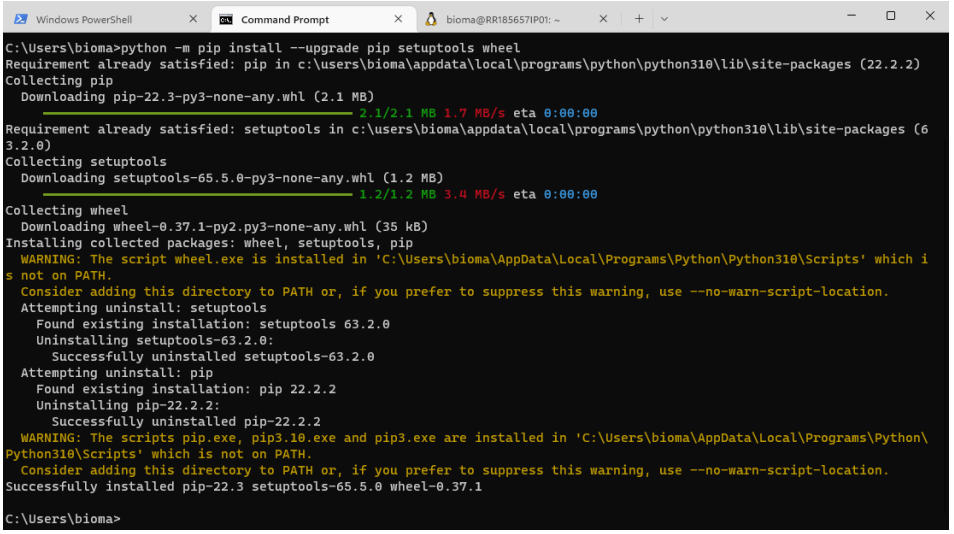

Figure 3.12-14: Terminal output of upgrading pip, setuptools, and wheel.

4. I like to have an easy-to-access folder for virtual environments, thus I always create a folder like `c:\penv` for this purpose. To create this folder, run the following command in the terminal: `mkdir c:\penv`. You can name this folder whatever you like. The importance of this will become evident in step 8. Also importantly, the folder in which you create your virtual environments does not need to be the folder in which your code resides.

5. Now **create** a virtual environment called venn (this is an example for a program about Venn diagrams; you can call it whatever name you prefer):

   ```
   python -m venv c:\penv\venn
   ```

6. To **activate** this environment execute the following command in the terminal in VSC:

   ```
   C:\penv\venn\Scripts\activate
   ```

   Now the command line will show the prompt named as the environment that was activated.

7. To **deactivate** a virtual environment, simply type `deactivate` in the command prompt.

8. The ideal solution is to use the ActivateEnv.bat file described in the appendix (add `c:\venv` to the PATH environment variable). Call it from a .bat or a .sh file as in the following example:

```
C:\Users\bioma>C:\penv\venn\Scripts\activate

(venn) C:\Users\bioma>
```

Figure 3.12-15: Command prompt showing the activated virtual environment.

```
ActivateEnv DiNet "C:\Dropbox\!JBGResearch\!!!DiNet\FOO_QtPy"
```

3.13 Troubleshoot

As reported by Lorena Roa de La Cruz, you could see an error when trying to activate a virtual environment. This is due to configuration of permissions. If this happens, run the command:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

After that, run the script `activate`.

```
PS C:\OpenAI> activate
activate : File c:\OpenAI\.venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled on this system.
At line:1 char:1
+ activate
+ ~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
PS C:\OpenAI> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
PS C:\OpenAI> activate
(.venv) PS C:\OpenAI>
```

Figure 3.13-16: PowerShell execution policy error and fix: running `Set-ExecutionPolicy` followed by `activate`.

3.14 Required (after 3.11 or 3.12): Install Python libraries

We will install libraries for the python environment for this project environment. The most important libraries are:

- OpenAI
- Anthropic
- Langchain
- PyQt5
- NumPy
- Matplotlib
- SciPy

- Jupyter
- Pandas

For each one of them type the following command: `python -m pip install [library name]`. For example:

```
python -m pip install NumPy
```

Alternatively, if you are using git bash, you can simply type:

```
pip install NumPy
```

Repeat for the libraries listed above. You should see something like this:

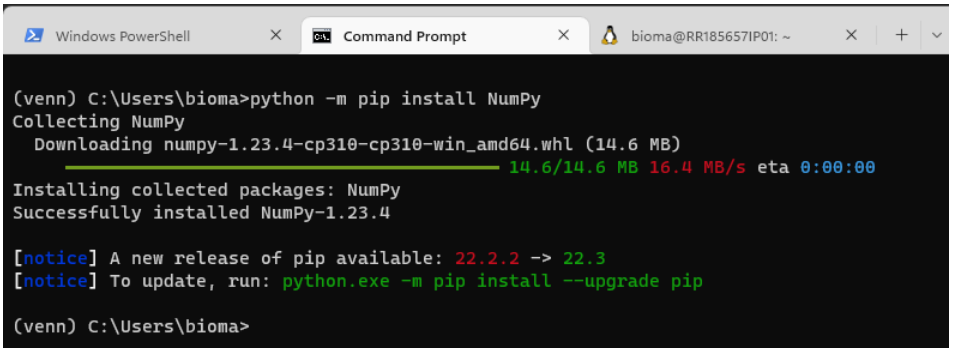

Figure 3.14-17: Terminal output showing a successful NumPy installation.

It is possible that the code fails; the source code repository changes over time. In that case the error will be something like:

```
ModuleNotFoundError: No module named 'library_name'
```

Where `'library_name'` stands for the library name that is causing the error. In that case, simply follow the procedures above to install the missing library.

An important suggestion that comes from experiencing frustration in the past is this: Keep the base environment clean, that is, with minimal libraries. Conflicts between libraries will arise once you install a number of them. This is why it is a good practice to create an environment for each project.

You know your environment is fully functional when you can execute the file agentGroq.py, agentGPT.py and/or agentClaude.py.

3.15 Required: Jupyter Notebook

Optional: Now, create a Jupyter Notebook.

1. Abundant relevant information regarding Jupyter in VSC is available at https://code.visualstudio.com/docs/datascience/jupyter-notebooks

2. Activate the environment you want to use according to step 3.12.f
3. You need to attach the kernel of your virtual environment. For example, if I have a virtual environment called venn:

```
python -m ipykernel install --user --name=venn
```

4. Execute step 3.12.a
5. In VSC, run the command `Create:New Jupyter Notebook` from the Command Palette (Ctrl+Shift+P) or by creating a new .ipynb file in your workspace.
6. You might receive a Security Alert regarding the firewall. Allow access.

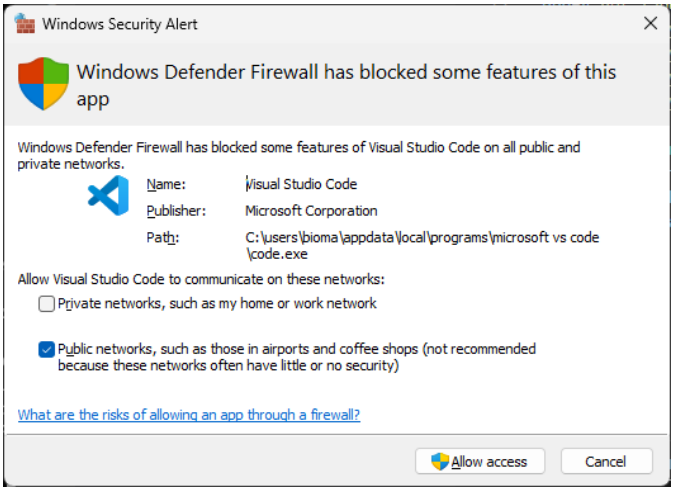

Figure 3.15-18: Windows Defender Firewall alert—allow access for Visual Studio Code.

7. After you enter a command, you might see the following message:

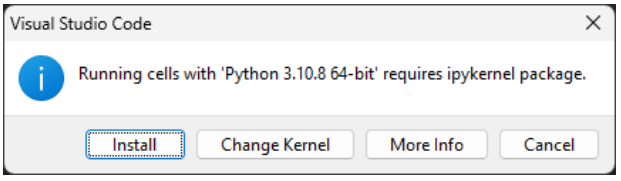

Figure 3.15-19: VS Code prompt to install the ipykernel package—click Install.

Install it.

3.16 Python commands in Terminal

Finally, you can simply enter Python commands in the Terminal. Open a Command Prompt and type the following command:

```
python
```

This will allow you to enter python commands, e.g. `1+1`

APPENDIX: ActivateEnv.bat

ActivateEnv.bat to be placed in `c:\venv` to activate Python environments

```
@echo off
setlocal

:: Read parameters
set "ENV_NAME=%~1"
set "BASE_DIR=%~2"
echo Starting activation of the virtual environment '\%ENV_NAME%'...

:: Validate input
if "%ENV_NAME%"==""  (
    echo [ERROR] Environment name not specified.
    goto :eof
)

if "%BASE_DIR%"==""  (
    echo [ERROR] Base directory not specified.
    goto :eof
)

:: Check if base directory exists
if not exist "%BASE_DIR%" (
    echo [ERROR] Directory not found: %BASE_DIR%
    goto :eof
)

:: Change to working directory
cd /d "%BASE_DIR%"

:: Define the path to the venv under C:\penv
set "VENV_ROOT=C:\penv"
set "VENV_ACTIVATE=%VENV_ROOT%\%ENV_NAME%\Scripts\activate.bat"

:: Check for the activation script
if exist "%VENV_ACTIVATE%" (
    echo calling activation...
    call "%VENV_ACTIVATE%"
    echo activation was called...
) else (
    echo [ERROR] Virtual environment activation script not found:
%VENV_ACTIVATE%
    goto :eof
)
```

```
echo Activation of the virtual environment '\%ENV_NAME%' has ended

echo Launching VSC Starting
code .
echo Launching VSC Ended

endlocal
```

Chapter 4

Introduction to Programming Environments

4.1 Introduction

There is a large number of programming languages. For the purpose of data analytics, we will distinguish two main families of languages: complied vs. scripting languages. They represent two different paradigms of programming language design, each with its own advantages and trade-offs. The key difference lies in when and how the source code you write is translated into machine code that the computer can understand and execute.

In the realm of data analytics, the choice between using a compiled language like C++ and a scripting language like Python often comes down to the specifics of the project at hand, including its complexity, required execution speed, and the need for direct hardware access.

4.1.1 Compiled Languages

Compiled languages are those where the source code is entirely translated into machine code before execution. This translation is done by a compiler, a program that takes the entire source code as an input and generates an executable file. Examples of compiled languages include C++, Java, Rust, Go, and Swift.

Compiled languages offer several important benefits including:

- Performance: Since the code is precompiled into machine code, compiled programs usually run faster and more efficiently than interpreted ones.
- Type checking: Many compiled languages have strict type systems, which catch many type-related errors at compile time, before the code is run.
- Security: The source code is not distributed with the software, which makes it harder for others to reverse engineer the software.

However, compiled languages also have some drawbacks:

- Development speed: The edit-compile-run cycle can be time consuming, particularly for larg software projects.
- Portability: The generated executable is usually specific to a type of processor and operating system. Availability for different operating systems or processors might require specific source code alterations and independent compilations.

Compiled languages like C++ are translated into machine code before execution, making them very fast because they're executed directly by the computer's hardware. This can be a significant advantage when dealing with huge datasets or when computation speed is a critical factor.

However, the syntax of languages like C++ is more complex and it may require more lines of code to achieve the same result as a scripting language. Moreover, C++ lacks many of the built-in data analysis functionalities found in languages such as Python, MATLAB or R, so it's necessary to use external libraries like Eigen or Armadillo for matrix operations and other complex computations.

4.1.2 Scripting Languages

Scripting languages, on the other hand, are typically interpreted from text into machine instructions, line-by-line of code just in time for execution. While this can make scripting languages slower than compiled languages, they're often easier to write and read because of their high-level syntax. Examples of scripting languages include Python, MATLAB, R, JavaScript, Ruby, and PHP.

Scripting languages offer several advantages:

- Ease of use: Scripting languages are usually easier to learn and use. They have less verbose syntax and more built-in functionality.
- Flexibility: Since they are interpreted line-by-line, it's easier to make changes to your code and see their effect immediately without having to recompile.
- Portability: The same script can be run on any system that has the correct interpreter installed, without needing to change the code.

Among the drawbacks of scripting languages there ar issues related to:

- Performance: Interpreted languages are generally slower than compiled languages because of the overhead of interpreting code line-by-line during execution.
- Dependency on the interpreter: The code can only be run on systems that have the correct interpreter installed.

Python is particularly well-suited for data analytics because it has numerous powerful libraries for data analysis (like Pandas, NumPy, and SciPy) and visualization (like Matplotlib and Seaborn). In addition, Python's simple, straightforward

syntax makes it an excellent choice for quick prototyping and exploratory data analysis.

It must be noted that even with all the benefits of Python, the simplified MATLAB syntax for matrix operations facilitates algorithm development since the source code closely resembles mathematical notation.

4.1.3 Comparative Examples

Let us consider a simple example where we want to multiply two matrices A and B. The language examples are listed below in alphabetical oder by the name of the language.

In C++:

```cpp
#include <Eigen/Dense>
#include <iostream>

int main() {
    Eigen::Matrix2f A;
    A << 1, 2,
         3, 4;
    Eigen::Matrix2f B;
    B << 5, 6,
         7, 8;
    std::cout << "The product AB is:\n" << A * B;
}
```

../source/IntroProgMatMul.c

In MATLAB:

```MATLAB
A = [1, 2; 3, 4];
B = [5, 6; 7, 8];
disp('The product AB is:')
disp(A * B)
```

../source/IntroProgMatMul.m

In R:

```R
A <- matrix(c(1, 3, 2, 4), nrow=2, ncol=2, byrow = TRUE)
B <- matrix(c(5, 7, 6, 8), nrow=2, ncol=2, byrow = TRUE)

print("The product AB is:")
print(A %*% B)
```

../source/IntroProgMatMul.r

In Python:

```python
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("The product AB is:\n", np.matmul(A, B))
```

../source/IntroProgMatMul.py

From the above example, it's clear that MATLAB code is much more concise and easier to understand than C++. However, if we were dealing with very large matrices or needed to perform this operation many times in a loop, the C++ code might run faster due to being a compiled language.

4.1.4 What is Python?

Python is an *interpreted high-level* programming language for general-purpose computing.

- *Interpreted* means that a program in Python has to be decoded line by line by another program called the Python interpreter and translated into something a computer can understand.
- *High-level* means that there is a strong abstraction of the elements of the hardware of the computer; for instance, memory does not need to be pre-allocated in Python to create a matrix, whereas the C language (a low-level language) requires it. You can think of the level of the language as how close the instructions are to the metal of the machine.
- The process of language interpretation is called *compilation*, that is, the translation of one computer language into another language. This could happen multiple times until the instructions are reduced to a language called *assembler* or *assembly*; this code matches closely the architecture of the computer executing the program. For instance, Java programs are compiled into a language called Java bytecode, which the Java Virtual Machine translates into assembler.

4.1.5 Python Installation

Refer now to the software installation guide provided in a separate document.

Being an interpreted language, it means that Python requires an *interpreter*. This is what you install when you "*install Python*". A Python program is simply a text file that contains instructions the interpreter can understand one line (or block of lines) at a time. Since Python is open source, it has many contributors; this results in a robust ecosystem, but the drawback is that there are numerous software libraries that have to be installed while tracking a complex web of co-dependencies. Hence, there is some difficulty in installing a working Python environment to conduct quantitative analysis.

An easy way to use the Python language is through an *Integrated Development Environment*. There are many IDEs that can execute Python code. Being familiar with more than one environment is important as each has strengths and weaknesses. Furthermore, it is a common occurrence that Python programs appear to fail in one environment but work properly in another; this, of course, is a matter of configuration, but it can be a source of frustration in absence of awareness of the potential sources of conflicts in software. Particularly, you want to know how

to use tools frequently used in software development, and you need exposure to the multiple options you will find in academia, government and industry.

4.1.6 Executing Python Commands

Commands in Python are case-sensitive, that is, you must respect the capitalization of instructions as they appear in the documentation. The Python interpreter offers multiple mechanisms to interact with the environment. The most commonly used is the Python command line. You can start it form a command shell. Simply type `python`. The command line promt will change to `>>>`

You can enter Python commands in the command line prompt. Press *Enter* or *return* to execute commands. Like most other programming languages, Python provides mathematical expressions. The building blocks of expressions are variables, numbers, operators, scripts and functions

The basic operations are listed below.

Symbol	Operation
$+$	Addition
$-$	Subtraction
$\star$	Multiplication
$/$	Division
$\star\star$	Power

Some commands are just like you would type things in a calculator. If you want to refer the result of the command, you need to give it a name, or in other words, assign the result to a variable. The following command assigns the result to the variable s. (Note that Python didn't save the true answer 7/12 but rather a decimal approximation.)

```
>>> s = 1/3 + 1/4
```

If you want to know what the variable s contains, simply type its name in the command prompt:

```
>>> s
0.5833333333333333
```

If a command does not fit on one line, use a backslash (\) followed by Return/Enter to indicate that the statement continues on the next line. For example,

```
>>> s = 1 - 1/2 + 1/3 -1/4 + 1/5 - 1/6 + 1/7 \
... :- 1/8 + 1/9 - 1/10 + 1/11 - 1/12
```

Type s and Enter to see the new value of s.

```
>>> s
0.6532106782106782
```

Use the double star($**$) to raise something to an exponent.

```
>>> 89**2
7921
```

Imaginary numbers can be used with the constant j, which stands for $\sqrt{-1}$ by default.

```
>>> (1j)**2
(-1+0j)
```

Hence, you must be careful in using the variable j when dealing with complex numbers. What do you expect from the following operation? Can you explain what you observe? You might need to complete the entire chapter before you can answer this question. Record your explanation as answer #6.

```
>>> j = 10
>>> j = j*(-1)**(0.5)
>>> j
```

Does the result stored in the variable `j` match your expectation? Explain. Record your explanation as answer #7.

Python uses conventional decimal notation, with an optional decimal point and leading plus or minus sign, for numbers. Scientific notation uses the letter e or E to specify a power-of-ten scale factor. Imaginary numbers use i as a suffix. (This course does not use complex numbers, but you might generate errors with one.)

Some examples of legal numbers are: 1, -1, 0.0001, 9.87654321, 1.2345e10, 1.2345*10**10, 1j, -2j, 3e5j.

When assigning names to variables or expressions, it is good practice to make the names useful. For example, in the second command above, we gave the result the name rhoSq to imply "rho squared". Observe that you **CAN NOT** have spaces in variable names (i.e. no spaces in names on the left hand side of the equals sign). Giving useful names to your variables or output will help you keep track of what you are doing.

You can use the up and down arrow keys to easily recall and then edit a command with the left and right arrows. Try it. This is the fastest way to repeat Python commands you might have used in the past.

4.1.7 The VSC Python Editor: Executing A Simple Program

Download the file `lab1SimpleInstruction.py`. Open the file in VSC. The following instructions will appear in the editor:

```
k=0;
for i in range(1,11):
    k=k+1/10;
    print(k)
print(k==1)
```

You can click the *Run file* button on the toolbar (as shwon in Figure 4.1-1), or press F5. We call all these actions *to execute a program*. The program `lab1SimpleInstruction.py` adds ten times the number 0.1. The comparison of the number 1 against the sum returns a value of *false*, whereas elementary

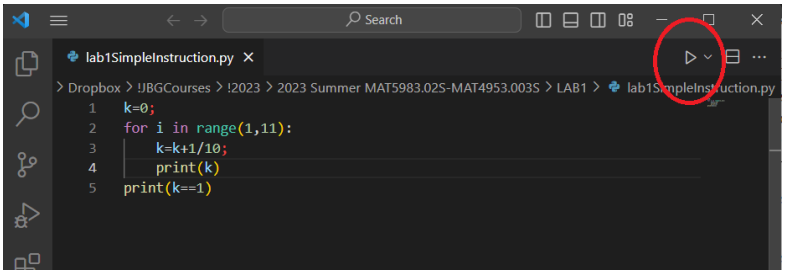

Figure 4.1-1: To execute a Python file in VSC, simply click on the Run button, represented by a triangle pointing to the right. It has been highlighted with a circle.

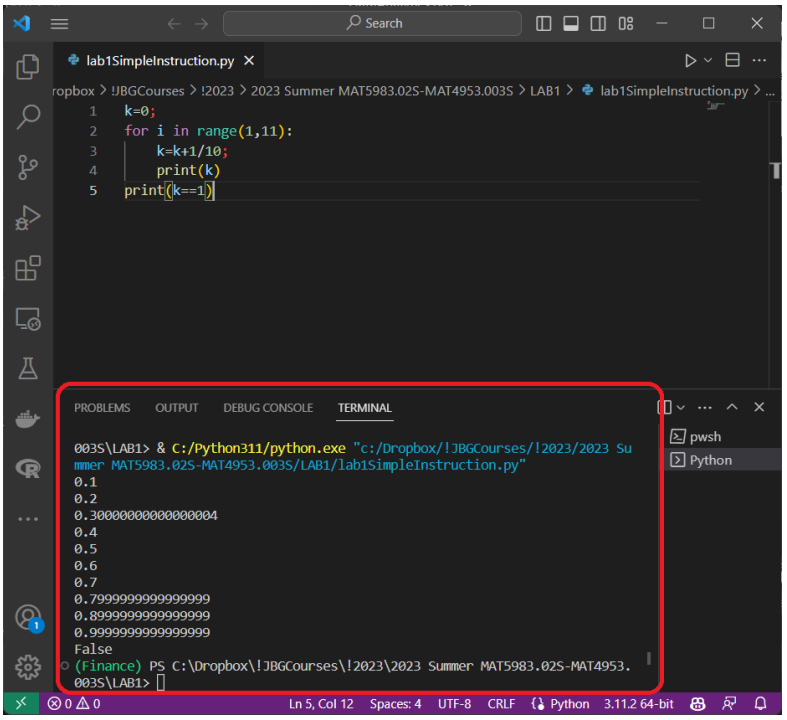

Figure 4.1-2: Executing a python program activates the command line. Results are printed in that panel. The command line is highlighted by the round box.

arithmetic tells us that adding the number 0.1 ten times should result in 1, and the returned value should be *true*. Why is this result false? Record your explanation as answer #8. Once the program is executed, the bottom portion of VSC shows a panel labeled TERMINAL, as well as other tabs labeled PROBLEMS, OUTPUT, and DEBUG CONSOLE, as shown in Figure 4.1-2

Note that you can enter commands in the terminal. For example, type `1+1` and hit enter. Try mathematical operations such as square root, or trigonometric functions. What challenges do you encouter? Record your explanation as answer #9.

In the program `lab1SimpleInstruction.py` all lines of code were executed sequentially, one at a time. This is what we call a **script file**. That is, when you execute a script file, you expect soemthing tangible to happen, whether it is a message on the screen, a file altereed, communication taking place, etc.

There is a way to use source code files to hold code that will be executed

later, without necessarily executing any instruction. At this pointm, open the file `lab1Function.py`. You will see the following instructions:

```
def sum1(n):
    # sum1(n) computes the sum of 1 + 2 + ... + n
    total = 0;
    for i in range(1,n):
        total = total + i
    return total
```

This file defines a function named `sum1(n)`. The command `sum(3)` returns 6 since $1 + 2 + 3 = 6$. It is instructive to see how this code does its work.

- The instruction `def` indicates the beginning of a *function*. Note that all instructions that are executed inside that function have a vertical alignment shifted to the right to indicate that all these commands are subordinate to `def`. This shift in vertical alignment is called indentation. It is part of the syntax of the Python programming language, therefore you must always treat indentations strictly.

- The instruction `# sum1(n)...` is a *comment*. Lines of code that start with the symbol `#` are not executed. The semicolon at the end is optional.

- The instruction `total = 0;` is an *assignment*. This line of code creates the variable `total` and assigns to it the value zero.

- The instruction `for i in range(1,n):` starts a *loop*. The subordinate instructions, identified by indentation, will be repeated n times. Particularly, the instruction `range(1,n)` creates a sequence of integers starting in 1 and ending in n. What is the outcome of `range(10,13)` ?

- The instruction `total = total + i` is an assignment. It is very important to emphasize that this is not an equation; if it were, `total` would cancel and `i` would be zero. What takes place is that for every repetition of the loop, the variable `total` is redefined to contain its previous value plus the value of `i`.

- The instruction `return total` is the output of the function. That is, if somehow we could execute this function like `x = sum1(3)`, the variable `x` would contain the value 6, because `sum1(3)` $= 1 + 2 + 3 = 6$.

Now, we will modify this file. Append at the end a program called `sum2` in which we sum integers squared. The next question has to be: How do we execute these functions?

The command `python [filename]` can be used to execute a Python program from the terminal, or any command line prompt. But if we run the following in the command prompt, nothing happens:

```
python lab1Function.py
```

Why? Record your explanation as answer #10.

Now, add the following line of code to the end of the file, **without indentation**:

```
print(sum1(2))
```

Execute the program. What was the result of `sum1(2)`? Is this what you expected? Why? Record your explanation as answer #11. Compute by hand what the output of `sum1(3)` should be. Now change the last line in the program to read `print(sum1(2))`

What we did was simply to define a function and then use it. Note that if you place the previous line of code at the beginning of the file, the Python interpreter would raise an exception because the function `sum1` is not known to the interpreter yet... keep in mind that the Python interpreter executes one line of code at a time in sequential order (perhaps with branching and repetition, as we will explore later). Therefore, functions must be defined before they can be invoked within a file.

Did the progeam prodice the result you were expecting? In case it did not, enter the following instruction in the terminal: `man range`. This command shows the manual (`man`) for the function `range`. The fact that `range` behaves in certain ways does not force you to abide by that particular design. In fact, you can modify `sum1` so that the outcome corresponds to what a naive user would expect. To achieve this, you will need to change the limit of range inside the function. Implement it and explain. Record your explanation as answer #12.

4.1.8 Executing Functions

In the previous section, we added a single line of code at the end of the program to compute the function we programmed. However, there is a more interesting use case: Create a function that other programs can reuse. In order to use an existing function from a file, we must import that function into the Python environment. We already did something like this when we imported NumPy.

We will now import the function `sum1` from the file `lab1Function.py`. To do this, execute the following command:

```
import lab1function as LAB1
```

This instruction loads into memory a directory of all functions present in that file. Now you can invoke the functions you created. For example,

```
LAB1.sum2(3)
```

It is also possible to load into memory a single function from a file. We can accomplish this by executing

```
from lab1function import sum1
```

In this case, there is no alias associated with the function. To use, you can simply call the function with a parameter, as in `sum2(3)`

Do some research on how to load multiple functions with a single instruction. Explain. Record your explanation as answer #13.

4.2 Introduction to NumPy

Numerical operations can be programmed in Python. A library called `NumPy` encapsulates much needed functionality and has become the *de facto* standard. The basic data element in `NumPy` is a multidimensional array. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a low-level language such as C.

For example, at the Python prompt type in the following command and press Enter.

```
import numpy as np
```

This command loads into memory a reference to the NumPy library. Now you can access all functions in this library by using the prefix `np`; this is an *alias*. You can pick any alias you want, but it is customary to use `np` for NumPy. Now enter the following command:

```
np.sqrt(9)
```

The function `sqrt` computes the square root of a real number. Here is an example, and the resulting values.

```
>>> rho = (1+np.sqrt(5))/2
>>> rhoSq = rho**2
>>> a = rhoSq + rho
>>> a
4.23606797749979
```

However, you cannot use the function `sqrt` with negative numbers. What would you have to do to alloy Python to compute the square root of a negative number and return a complex number? Record your explanation as answer #14.

The library NumPy offers easy access to commonly used constants, such as:

Constant	Value
`np.e`	Returns the number e
`np.pi`	Returns the number π
`np.inf`	Returns the number ∞
`np.nan`	NaN, not-a-number

Expressions use the arithmetic operators and precedence rules you already know by now. There are also functions, such as cosine or sine. You surround the input to the function by parenthesis. With NumPy, we can now access commonly used mathematical functions. Among others:

Function	Operation
`np.abs(x)`	Absolute value $\|x\|$
`np.sqrt(x)`	Square root $\sqrt{x}$
`np.exp(x)`	Exponential function e^x
`np.log(x)`	$\log(x)$ where x is positive real number
`np.sin(x)`	$\sin(x)$ where x is assumed to be in radians
`np.cos(x)`	$\cos(x)$ where x is assumed to be in radians

The library NumPy has *many* functions. A comprehensive list can be found at `https://numpy.org/doc/stable/reference/`. Now enter the following command:

```
x = np.random.standard_normal(3)
```

There is no output. Why do you think this is the behavior? Record your explanation as answer #15. On the right-hand side of the Spyder window there is a tab called *Variable Explorer*. It will show the value of x. Alternatively, you can enter

```
print(x)
```

Alternatively, you can print the result directly without assigning it to a variable. Try

```
print(np.random.standard_normal(3))
```

Why are the results of this last operation different to the values stored in variable x? Record your explanation as answer #16.

4.2.1 Matrix Operations

The basic data element in NumPy is a multi-dimensional array. Special meaning is sometimes attached to 1-by-1 matrices, which are called scalars, and to matrices with only one row or column, which are called vectors. Python has other ways of storing both numeric and non-numeric data, but in the beginning, it is usually best to think of everything as a matrix.

Row vectors are delimited in notation by square brackets. A comma separates individual numbers in a row vector. Multiple row vectors of the same size are separated by commas, and are delimited by square brackets to define a matrix. It is common practice to use a capital letter when naming matrices in order to distinguish them from a single vector, scalar or variable value.

Now, we will create a vector and a matrix to demonstrate simple matrix operations

Function	Operation
`x = np.array([1,2,3,4])`	Creates the row vector $\mathbf{x} = [1, 2, 3, 4]$
`b = np.array([1,2,3])`	Creates the row vector $\mathbf{b} = [1, 2, 3]$
`A = np.array([[1,2,3],[4,5,6],[7,8,9]])`	Creates the matrix $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
`B = np.ones([2,3])`	Creates the matrix $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
`C = np.zeros([2,3])`	Creates the matrix $A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

The following commands demonstrate basic vector manipulation cases:

Operation	Outcome
`x[-1]`	Returns 4, the last element of x
`x[1]`	Returns 2, the second element of x
`x[-2]`	Returns 3, the second to last element of x

The colon (:) operator is notable. In vector notation, it indicates a range of rows or columns. In Python, the first element of a vector has index 0; the second element has index 1, and so on. The colon operator $a : b$ starts at the row or column of index a and covers up to an index less than b. Thus, $A[0 : 2, 0 : 2]$ returns a matrix composed of rows 1 through 2, and columns 1 through 2 with respect to the matrix A.

The following commands demonstrate basic matrix manipulation cases:

Operation	Outcome
`A[1,2]`	Value of 2st row and 3nd column of A
`A[0:2,0:2]`	Returns $A = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
`A[1:3,1:3]`	Returns $A = \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$

The matrix A has three rows and three columns. Then, why does $A[1 : 100, 1 : 100]$ not produce an error? Record your explanation as answer #17.

The size or dimension of a matrix refers to the number of rows and number of columns a matrix has. The `shape` command returns the number of rows and columns of a matrix. Notice that the result is a vector - a one-dimensional matrix with 2 values where the first is the number of rows and the second value is the number of columns. By accessing each element of this vector, we can access each dimension individually.

Using the matrix A of our previous examples,

```
>>> y = A.shape()
>>> y
(3,3)
>>> y[0]
3
>>> y[1]
3
```

The following example will illustrate a subtlety in copying matrices:

```
>>> D = A

array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> D[0,0] = 9
>>> A

array([[9, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Note that by updating a value in D, the corresponding value in A was changed. Why? What would you need to do to prevent an update in D to change A? Record your explanation as answer #18.

The basic matrix operations are listed below, using the matrices in the previous examples.

Operation	Outcome
$A.T$	Transpose of A.
$A.conj().T$	Conjugate transpose of A.
$A@B.T$	Matrix multiplication $A \cdot B$.
$A \star A$	Element-wise multiplication
A/A	Element-wise division
$A \star \star 2$	Element-wise exponentiation

NumPy gives an easy way of solving systems of linear equations, Given the matrix A and the vector $\mathbf{b}$ from the previous examples, the linear system $A\mathbf{x} = \mathbf{b}$ can be solved by invoking `np.linalg.solve` as follows

```
>>> x = np.linalg.solve(A,b)
array([-0.23333333,  0.46666667,  0.1       ])
```

It is easy to use Matrix multiplication $A\mathbf{x}$ to check the answer

```
>>> A@x
array([1., 2., 3.])
```